

北 京 邮 电 大 学

《编译原理与技术课程设计》

报 告

指导教师：_____王雅文_____

姓名	班级	学号	备注
周正明	2022211309	2022211280	组长/llvm 分支
庞博	2022211307	2022210466	main 分支
许恒栋	2022211307	2022211209	tran 分支
周宇洋	2022211307	2022210470	main 分支
范兼玮	2022211307	2022212442	tran 分支
万云杰	2022211307	2022211183	撰写文档

计算机学院（国家示范性软件学院）

2025 年 5 月 12 日

目录

1	课程设计的任务和目标	3
1.1	任务要求	3
1.2	课程目标	3
2	需求分析	4
2.1	开发环境	4
2.2	功能分析	4
2.2.1	词法层面	4
2.2.2	语法层面	4
2.2.3	语义层面	5
2.3	编译器功能模块需求	5
2.3.1	词法分析器	5
2.3.2	语法分析器	5
2.3.3	语义分析器	5
2.3.4	中间代码生成器	5
2.4	输入输出需求	6
2.4.1	输入	6
2.4.2	输出	6
2.5	其他需求	6
3	总体设计	7
3.1	总体计划	7
3.2	分支模块设计	7
3.2.1	tran 分支	8
3.2.2	llvm & main 分支：生成与优化	8
4	详细设计	9
4.1	LLVM 分支	9
4.1.1	准备工作	9
4.1.2	词法分析	10
4.1.3	生成 LR 分析表（开发阶段）	10

4.1.4	语法分析 (基于 LR 表)	10
4.1.5	翻译方案 & 语义分析	10
4.1.6	代码优化	11
4.1.7	模块链接	11
4.1.8	目标代码生成	11
4.2	tran 分支	11
4.2.1	词法分析	12
4.2.2	语法分析	13
4.2.3	语义分析	16
4.2.4	C 代码生成	17
4.2.5	错误处理设计	17
4.2.6	编译与运行设计	17
4.3	main 分支	18
5	源程序清单	18
5.1	LLVM 分支源程序清单	18
5.2	tran 分支源程序清单	20
5.3	main 分支源程序清单	22
6	程序测试	22
6.1	正确的测试用例	22
6.2	错误检测与处理	23
7	课程设计总结	25
7.1	体会与收获	25
7.2	设计过程中遇到或存在的主要问题及解决方案	25
7.3	改进建议	25

Pascal-S 语言编译程序的设计与实现

1 课程设计的任务和目标

1.1 任务要求

课程设计的目标是设计一个针对 Pascal-S 语言的编译程序，使用 C 语言作为编译器的目标语言。

课程设计的目标是设计并实现一个编译器，该编译器能够将 Pascal-S 语言编写的源代码转换为 C 语言代码。Pascal-S 是 Pascal 语言的一个子集，专门用于教学目的，它包含了 Pascal 语言的核心特性，但去除了一些复杂的构造以简化学习和编译过程。

编译器的设计将分为几个主要部分：词法分析、语法分析、语义分析和代码生成四大核心模块。

1. **词法分析 (Lexical Analyzer)**: 该部分将读取源代码，并将其分解成一系列的标记 (tokens)，这些标记是编译过程中语法分析的基本单位。
2. **语法分析 (Syntax Analyzer)**: 语法分析器将使用词法分析器提供的标记来构建抽象语法树 (AST)。AST 是源代码的树状表示，反映了程序的结构。
3. **语义分析 (Semantic Analyzer)**: 语义分析器将检查 AST 以确保源代码的逻辑是一致的，例如变量的声明与使用是否匹配，类型是否兼容等。
4. **代码生成 (Target Code Generator)**: 代码生成器将遍历语义分析后的抽象语法树 (AST)，并将其转换为目标语言的代码（如 C 语言）。该模块负责将程序结构、表达式、控制语句等翻译为目标语言的等价表示，并保持语义的一致性。

1.2 课程目标

1. 深入理解编译程序的工作原理和各阶段的核心算法。
2. 掌握词法分析器、语法分析器的设计方法，以及语义处理和中间代码生成的实现技术。
3. 提升编程能力和复杂系统的设计能力，培养团队协作与问题解决能力。

2 需求分析

2.1 开发环境

在本次课程设计中，我们没有使用编译原理课程设计的传统工具：lex 和 yacc，而是决定使用 C++17 手动实现词法分析和语法分析。

为了撰写开发文档和实验报告，我们利用了 Overleaf 和腾讯文档的在线文档协作功能。这使得文档的共享和协作变得更加高效和便捷，尤其是在团队分布在不同地点时。

在代码管理方面，我们使用了 Git 作为版本控制工具，并托管项目至 GitHub 上进行协作开发。Git 使得我们能够对代码的每一次修改进行精确记录和回溯，便于版本管理与问题追踪；同时，GitHub 提供了良好的远程仓库托管平台，使团队成员可以方便地进行代码拉取、提交、合并与审阅。在多人协作过程中，我们通过创建分支、发起 Pull Request 和 Code Review 等机制，有效避免了冲突并提升了代码质量。

2.2 功能分析

2.2.1 词法层面

识别保留字（如 `program`、`var`、`begin`、`end`、`if`、`then`、`else`、`while`、`do`）、标识符、常量（整数、实数、字符串）、运算符（`+`、`-`、`*`、`/`、`=`、`<`、`>` 等）和界符（逗号、分号、括号等）。

2.2.2 语法层面

支持基本的程序结构，包括：

- 程序定义：`program` 标识符 ；
- 变量声明：`var` 标识符列表 ；
- 语句块：`begin` 语句列表 `end`
- 条件语句：`if` 表达式 `then` 语句 [`else` 语句]
- 循环语句：`while` 表达式 `do` 语句
- 赋值语句：标识符 `:=` 表达式

2.2.3 语义层面

在语义层面，主要完成以下检查：

- 变量是否已声明
- 类型一致性检查（如赋值语句左右类型匹配）
- 表达式运算合法性（例如除数不能为零等）

2.3 编译器功能模块需求

2.3.1 词法分析器

- 将源代码分割为 token 流，记录每个 token 的类型和值
- 过滤空格、注释等无效字符
- 处理非法字符等词法错误

2.3.2 语法分析器

- 使用递归下降或 LR 分析算法，构建语法树或验证语法结构
- 报告语法错误（如括号不匹配、语句结构错误）

2.3.3 语义分析器

- 建立符号表，记录变量的名称、类型与作用域
- 检查语义错误（如未声明变量、类型不匹配）

2.3.4 中间代码生成器

- 将语法树转换为中间表示（如三地址码、AST 或 P-code）
- 实施基本优化（如常量折叠、公共子表达式消除）

2.4 输入输出需求

2.4.1 输入

支持合法的 Pascal-S 源代码文件作为输入。

2.4.2 输出

- 词法分析阶段：输出 token 列表
- 语法分析阶段：输出语法树或语法正确性报告
- 语义分析阶段：输出符号表信息及语义错误提示
- 中间代码生成阶段：输出中间代码（如三地址码）

2.5 其他需求

- 支持错误处理与提示，帮助用户快速定位问题代码
- 代码结构清晰，模块间低耦合，便于后续调试与维护

3 总体设计

3.1 总体计划

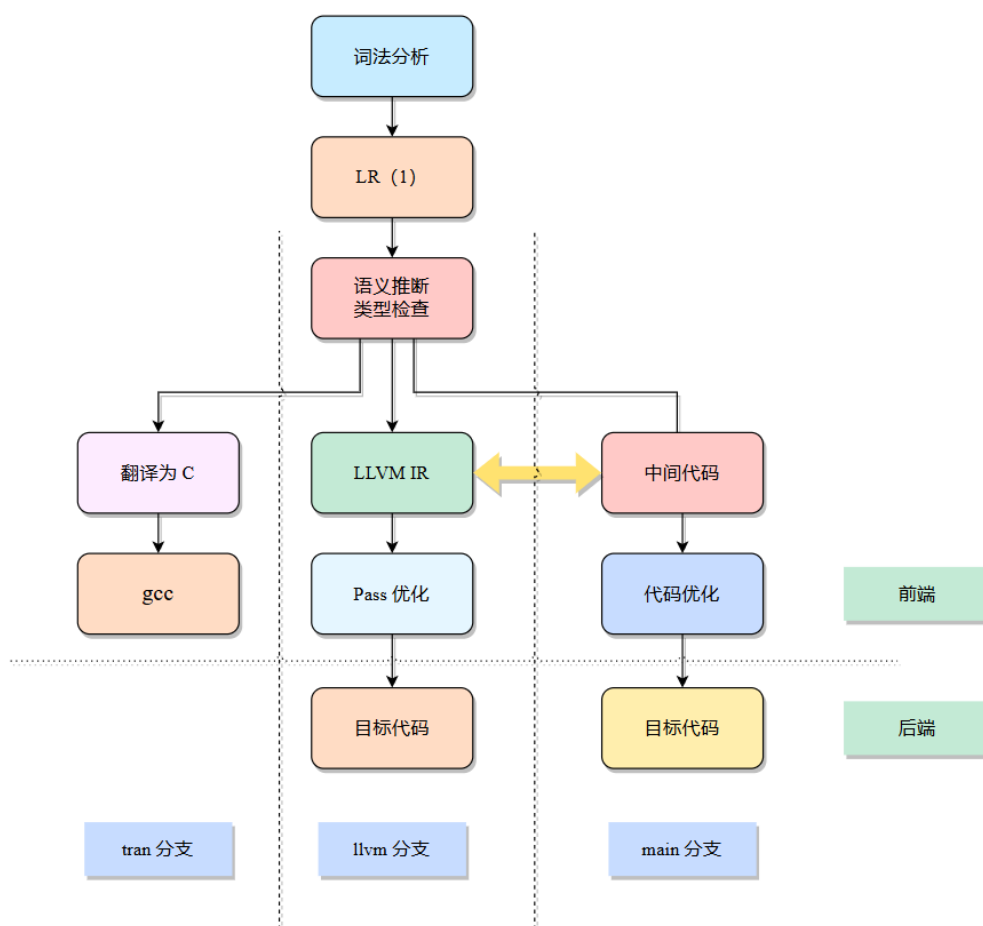


图 1: 总体设计图

总体计划如下:

1. 实现 Pascal-S 到 C 的代码翻译
2. 手工解析 Pascal-S 子集语法, 并基于 LLVM 构建 IR 表达
3. 尝试手工实现各主要编译模块, 以深入理解编译器构造

3.2 分支模块设计

项目开发分为多个功能分支, 分别聚焦于不同阶段的实现。

3.2.1 tran 分支

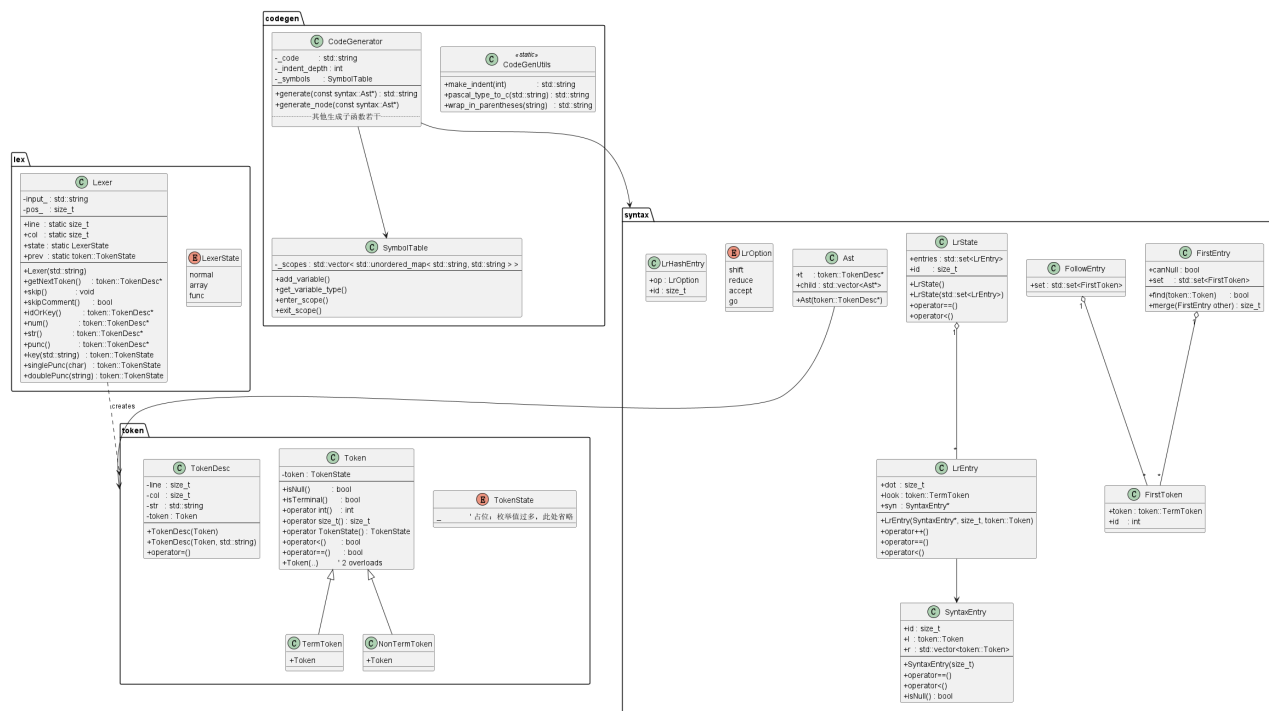


图 2: tran 分支类图

- **项目目标：**实现 Pascal-S 子集到 C 的完整编译链路
- **技术意义：**掌握编译器后端工作原理，深入理解代码生成过程
- **功能范围：**支持变量声明、赋值语句、控制流结构（if、while）、输入输出语句（如 writeln），生成符合 C99 标准的代码

3.2.2 llvm & main 分支：生成与优化

该部分以 LLVM 框架为核心，构建从 AST 到 LLVM IR 的完整生成与优化流程，主要包括以下几个层级：

(1) 前端接口层

- 接收 Pascal-S 的抽象语法树（AST）作为输入
- 定义统一接口以对接语法分析器，建立 AST 节点与 LLVM IR 元素之间的映射规范

(2) LLVM IR 生成层

- 核心模块：完成 AST 到 LLVM IR 的逐条转换
- 包含以下子模块：
 - 作用域管理模块：支持嵌套作用域及符号表管理
 - 类型映射模块：将 Pascal-S 类型映射为 LLVM 类型（如 `integer` $\rightarrow$ `i32`）
 - 指令生成模块：按语法结构生成相应的 LLVM IR 指令（如赋值、条件跳转等）

(3) 优化管理层

- 集成 LLVM 自带的优化 Pass（如死代码消除、常量传播等）
- 支持用户定义优化策略，便于扩展
- 提供优化级别配置接口，兼容 LLVM 的 `-O0/-O1/-O2/-O3` 等参数

(4) 代码生成层

- 支持多种输出格式，便于调试和部署：
 - 原始 LLVM IR 文件（`.ll`）
 - 优化后的 LLVM IR 文件
 - 二进制目标代码（通过 LLVM 后端生成 `.o` 或可执行文件）

4 详细设计

本系统包含三个主要开发分支：LLVM 分支、tran 分支、main 分支，对应现代化的中间代码编译链路，传统的 Pascal 到 C 翻译流程，中间代码优化。以下分别从结构、流程、数据结构和模块接口进行详细说明。

4.1 LLVM 分支

4.1.1 准备工作

- **Token 编码**：使用枚举类型表示所有词法 Token。
- **语法规则编码**：使用 `SyntaxEntry` 结构体记录产生式左右的 Token 序列。

4.1.2 词法分析

逐字符处理输入文件，根据首字符判断 token 类型，核心处理逻辑如下：

- 跳过注释
- 变量或关键字：以字母开头，判断是否为关键字，否则为变量；同时识别函数名
- 数字：判断整数或浮点数，支持数组索引
- 字符：以引号开头，识别为字符常量
- 标点符号：区分单字符与双字符运算符

所有识别结果存入双端队列，便于语法分析阶段使用。

4.1.3 生成 LR 分析表（开发阶段）

1. 标记 Nullable Token
2. 生成 FIRST 与 FOLLOW 集合
3. 构建 LR 状态转移图
4. 构建 LR 分析表并序列化保存

4.1.4 语法分析（基于 LR 表）

- **Shift**：移进当前符号，压栈
- **Reduce**：根据产生式规约，弹栈并执行对应语义动作
- **Accept**：匹配成功，完成语法分析

4.1.5 翻译方案 & 语义分析

在 Reduce 阶段同步执行语义动作，支持如下构造：

- 类型推导与检查
- 变量与作用域管理

- 表达式计算转换
- 函数定义与调用支持

4.1.6 代码优化

内置优化 Pass:

- 函数级: InstCombinePass, ReassociatePass, GVNPASS, SimplifyCFGPass
- 模块级: LoopAnalysisManager, CGSCCAnalysisManager

自定义优化:

- RemoveTerminatorPass: 去除冗余终止符

4.1.7 模块链接

支持 `external` 关键字声明外部符号。每个源文件生成一个 Module，最后统一链接生成完整目标代码。

4.1.8 目标代码生成

调用 LLVM 后端生成 x86-64 目标代码。支持输出原始 IR、优化 IR 以及可执行文件。

4.2 tran 分支

该分支实现 Pascal 子集 $\rightarrow$ C 源码的完整转换流程。

模块	主要功能
词法分析 (Lexer)	将源代码文本转换成 Token 流
语法分析 (Parser)	根据 Pascal 语法规则，生成抽象语法树 (AST)
语义分析 (SymbolTable)	检查变量、常量、函数的定义与使用合法性
代码生成 (CodeGenerator)	将 AST 转换为对应的 C 语言源代码

表 1: tran 分支各模块功能简述

4.2.1 词法分析

词法分析器采用面向对象的实现方式，核心类为 `Lexer`，其职责是对输入的 Pascal-S 源代码进行逐字符扫描并构造记号序列 (`TokenDeque`)。在内部数据结构设计上，词法单元由 `TokenDesc` 结构体描述，其中包含行号、列号、源码片段及对应的 `TokenState` 枚举值，实现了位置精确的错误定位与友好的调试输出。

(1) 状态机机制 词法分析器通过枚举 `LexerState` 维护三个工作状态：`normal` 为默认识别状态；`array` 用于解析数组下标时的特殊数字处理；`func` 用于函数 / 过程声明期间对标识符的额外登记。静态成员 `line`、`col` 与 `pos_` 用来实时跟踪当前扫描位置。

(2) 关键映射表 为提升匹配效率，系统在类私有部分维护三张散列表：`keyMap` 将关键字字符串映射到终结符；`puncMap` 负责单字符界符的快速识别；`opMap` 保存多字符运算符的集合；同时另设 `idfMap` 记录已识别出的函数和过程标识符，为语义分析阶段提供支持。

(3) 主循环流程 词法分析过程由 `getNextToken()` 驱动：首先调用 `skip()` 跳过空白与换行，再通过 `skipComment()` 处理 `{ }` 与 `//` 两种注释；随后依据当前字符分派至 `idOrKey()` (识别关键字 / 标识符)、`num()` (整型与实型常量并兼容数组索引)、`str()` (单引号字符串) 或 `punc()` (界符与运算符)。每当一个记号被识别完成，`line`、`col` 与 `pos_` 同步更新，确保后续 `Token` 的位置信息准确。

(4) 错误处理策略 为保证健壮性，`Lexer` 内部实现了多处异常检测：当注释未闭合、字符串缺少右引号、出现非法字符或数字格式不合法时，立即生成带位置信息的错误 `Token`，并终止进一步扫描；在高层调用 `lex()` 或 `getTokens()` 时即可统一捕获并报告。

(5) 接口与集成 词法模块向外暴露三组函数：`lex()` 支持直接传入文件名或字符串并一次性生成全局 `Token` 双端队列；`getNextToken()` 提供按需获取单个 `Token` 的能力；`printTokens()` 可将当前队列里全部 `Token` 以 `tokenNames` 字符串表映射打印，便于调试。语法分析器只需从 `token::getTokens()` 中持续取出 `Token`，即可完成词法—语法阶段的解耦。

支持的 Token 类型：

1. **关键字 (Keywords)** : `program`, `procedure`, `function`, `var`, `const`, `begin`, `end`, `if`, `then`, `else`, `while`, `for`, `to`, `do`, `of`, `read`, `write` 等；
2. **标识符 (Identifiers)**: 由字母或下划线开头，可包含字母、数字与下划线，大小写不敏感；

3. **常量 (数字、字符、布尔)**: 支持整数、实数、字符常量及布尔字面量 `true/false`;
4. **字符串 (Strings)**: 单引号包裹的字符序列;
5. **运算符 (Operators)**: 算术运算符 (+、-、*、/)、关系运算符 (<、>、<=、>=、=、<>)、逻辑运算符 (and、or、not) 以及整除 / 取模运算符 (div、mod);
6. **赋值符号 (Assignment)**: `:=`;
7. **界符 (Punctuation)**: 括号 ()、方括号 []、花括号 { }, 以及逗号、冒号、分号、句点等。

实现结构:

1. **状态管理**: 词法分析器维护三种工作状态: 普通识别 (`normal`)、数组索引处理 (`array`) 与函数声明处理 (`func`)。
2. **Token 映射表**: 包含关键字映射表 (`keyMap`) 以及用于匹配单字符和双字符运算符 / 界符的内部散列表 (由 `singlePunc()` 与 `doublePunc()` 调用 `CharMap/StrMap` 实现)。
3. **位置信息跟踪**: 每个 Token 保存行号、列号和全局偏移, 支持精准报错。
4. **特殊处理逻辑**:
 - 识别 `{}` 与 `//` 两种注释并跳过;
 - 对数组下标中的数字进行专门识别;
 - 在函数 / 过程声明阶段自动将标识符加入符号集合供语义分析使用。

4.2.2 语法分析

语法分析器承担着把词法阶段产生的 Token 流转换为抽象语法树 (AST) 的核心任务, 同时在编译期间完成语法合法性判定与错误上报。本项目采用 ****LR(1) 语法分析**** 方案: 先离线构造 LR(1) 状态机及动作表 (`LrTable`), 运行时再按 `Shift / Reduce / Accept / Goto` 四种动作驱动状态栈与符号栈, 实现自底向上的归约。其主要组件与流程概述如下。

1 文法描述与产生式管理 所有产生式统一存放于 `SyntaxEntry` 对象数组 (`SyntaxArray`) 中。每条产生式记录左部符号 `l`、右部符号序列 `r` 及唯一编号 `id`, 并通过静态函数 `initSyntaxes()` 在程序启动时一次性初始化。项目共 111 条产生式, 其中 `id=0` 为增强文法的开始符号 `S' → prog \$`。

2 First/Follow 集与 LL 支持 文件 `SyntaxLl.cpp` 实现了标准的 First / Follow 集搜索算法:

`searchNull()` 标记可推出空串的非终结符;

`searchFirst()` 递增式合并 First 集, 直至收敛;

`searchFollow()` 利用 First 和 Null 信息推导 Follow 集。

计算结果既用于 LL 调试, 也为 LR 构造中的 *lookahead* 推导提供依据。

3 LR(1) 状态机生成 `SyntaxLr.cpp` 通过典型的 闭包 + *Goto* 迭代算法建立 LR(1) 自动机:

`LrEntry` 记录形如 $\langle A \rightarrow \alpha \bullet \beta, a \rangle$ 的项目;

`LrState` 保存项目集并赋唯一 `id`;

`EntrySet` / `LrSet` 使用 `std::set` 去重;

`searchEntry()` 递归扩展闭包, `searchStates()` 按 Token 枚举 *Goto* 生成新状态并打印构造进度;

`fillLrTable()` 依据状态-迁移关系填充 `LrTable`, 同时解决 shift/reduce、reduce/reduce 冲突 (当前文法无冲突)。

为避免每次编译都重新建表, `Serialize.cpp` 提供二进制持久化: `serializeLrTable()` 与 `deserializeLrTable()` 将 `LrTable` 存取于 `lrTable.bin`, 运行时首先尝试加载, 若不存在再重建并缓存。

4 运行时分析驱动 `SyntaxCheck.cpp::lrCheck()` 是解析主循环:

$$\text{stateStack} \xleftarrow{\text{push}(0)} \quad \text{while } \neg \text{accept} \{ \}$$

读取栈顶状态 s 与输入符号 a ;

查询 `LrTable[s][a]` 得动作;

shift: Token 入符号栈 (`waitTokens`) 并推入新状态;

reduce: 按产生式长度弹栈, 调用 `semantic::callReduce()` 执行语义动作, 同时借助 `reduceAst()` 把对应子树组合为新 AST 结点再入栈;

accept: 设置 AST 根 (`setAstHead()`) 并结束循环;

遇未定义表项立即触发 `printError()`——输出错误 Token 的行、列与字面量, 帮助调试。

5 抽象语法树构造 AST 结点由 `Ast` 结构体持有指向词法 Token 的指针及可变长子结点数组, 所有正在归约的子树暂存于 `AstStack`。在规约动作里调用 `reduceAst(TokenDesc*, n)`: 新建父结点并弹出 n 棵子树逆序挂接; 最终 `getAstHead()` 返回完整 AST, 供语义分析与代码生成阶段使用。

6 关键对外接口

`syntax::lr::initLr()` ——自动加载或构造 LR 表, 分析前调用;

`syntax::lr::lrCheck()` ——语法检查 + AST 构建主入口;

`syntax::getAstHead()` ——获取归约完成后的 AST 根指针;

`syntax::ll::printFirst()/printFollow()` ——调试输出 First / Follow 集;

`syntax::lr::saveTable()/loadTable()` ——LR 表序列化接口。

7 错误处理与调试辅助 语法分析阶段的典型错误（非法 Token 顺序、缺失分号 / 括号等）一旦定位，通过栈顶状态和读头符号即可给出 “*possible token is wrong*” 提示，并附带精确行列信息。内部函数 `printDeque()` 在调试模式下会实时显示移进 / 待归约序列，便于开发者观察 LR 步骤。

综上，本语法分析模块以完整的 LR(1) 自动机为核心，结合高效的状态机持久化与实时 AST 构造，能够在保持线性时间复杂度的同时提供友好的错误诊断与调试手段，为后续语义分析与代码生成阶段奠定了坚实基础。

4.2.3 语义分析

语义分析阶段负责在**语法正确**的 AST 上进一步验类型一致性、作用域规则、常量属性等语义约束，并为后续代码生成附加类型与值信息。本项目采取 LR 规约触发方案：当语法分析器进行一次产生式规约时，同时调用绑定在该产生式上的语义动作（即 “Reduce-Action”），从而实现语义检查与 AST 构建的紧耦合。

1 核心组件 语义子系统由三类对象组成：

规约表 (ReduceTable) 单例 `unordered_map<size_t, ReduceFn>`，键为产生式 id，值为回调 `void (*)(token::TokenDesc*)`；初始化阶段一次性注册；

符号表 (SymbolTable) 采用“哈希桶 + 作用域栈”策略，支持 `add_variable`、`get_variable_type`、`enter_scope` 等接口，条目记录 `{name, type, isConst, dims}`；

属性增强 AST 每个结点通过 `attr` 字段缓存 `type`、`isConst`、`value` 等语义信息，供后端直接查询。

2 工作流程 当 LR 分析器规约 $\langle A \rightarrow \alpha \rangle$ 时，驱动函数 `TokenDesc* callReduce(id, lhs)` 依次完成：在 `ReduceTable` 中查找回调并执行；回调自 AST 栈弹出 $|\alpha|$ 个孩子结点，完成类型推断、符号表维护与常量折叠；生成新的 `TokenDesc`（其 `attr` 已填充）并重新压栈。若产生式无专属语义动作，则直接返回一个“占位” `TokenDesc` 以保持栈平衡。

3 关键检查点

类型检查 赋值、算术 / 逻辑运算、实参—形参、数组下标等处验证兼容性；不匹配时抛出 “Type-Mismatch” 错误并将结点类型标为 `error` 继续分析；

作用域分析 每遇见 `begin/procedure` 进入新层级，离开时 `exit_scope`；查找失败立即报告“Undeclared-Identifier”；

重复声明 `add_variable` 在当前层检测同名冲突（含函数 / 过程名），冲突则记录“Redefinition”；

控制流合法性 `if/while` 条件需为 `boolean`；`for-loop` 计数变量不可在循环体内再次赋值。

4.2.4 C 代码生成

功能：

- 遍历 AST，生成等价 C 代码
- 处理变量声明、表达式、控制语句翻译

结构与接口：

- `CodeGenerator::generate()`：总入口
- `generateNode()`：遍历树结构
- `generateExpression()`：表达式生成

4.2.5 错误处理设计

本系统实现了各阶段的错误定位与提示：

- 词法错误：非法字符、注释未闭合、字符串未闭合等
- 语法错误：缺失符号、括号不配、语序错误等
- 语义错误：未声明符号、重复定义、类型不匹配等

4.2.6 编译与运行设计

- 支持 C++17
- 项目结构清晰，头文件与源文件分离
- `main.cpp` 为主入口，控制整体流程：

1. 读取 Pascal 源文件
2. 词法分析 → 语法分析 → 构建 AST
3. 语义分析 → C 代码生成 → 输出文件

4.3 main 分支

5 源程序清单

5.1 LLVM 分支源程序清单

```
.  
  .gitignore  
  LICENSE  
  Pascal-llvm.sln  
  Pascal-llvm.vcxproj  
  README.md  
  project_structure.txt  
  doc/  
    syntax.md  
  include/  
    cmd.h  
    lex/  
      Lexer.h  
      Token.h  
    pass/  
      RmTerminatorPass.h  
    semantic/  
      Ast.h  
      ExpIR.h  
      FuncIR.h  
      Reduce.h  
      SymbolTable.h
```

```
TypeIR.h
VarIR.h
syntax/
    Serialize.h
    SyntaxCheck.h
    SyntaxEntry.h
    SyntaxLl.h
    SyntaxLr.h
src/
    cmd.cpp
    main.cpp
    lex/
        Lex.cpp
        Lexer.cpp
        Token.cpp
    pass/
        RmTerminatorPass.cpp
    semantic/
        Ast.cpp
        ExpIR.cpp
        FuncIR.cpp
        Reduce.cpp
        SymbolTable.cpp
        TypeIR.cpp
        VarIR.cpp
    syntax/
        Serialize.cpp
        SyntaxCheck.cpp
        SyntaxEntry.cpp
        SyntaxLl.cpp
        SyntaxLr.cpp
```

```

misc/
  lrTable.bin
  link_set/
    add.pas
    main.pas
  open_set/    ← 测试用例集
    00_main.pas
    ...
    69_matrix_tran.pas
output/
  output.ll
  output.o

```

5.2 tran 分支源程序清单

```

.
.gitignore
CMakeLists.txt
lrTable.bin
main.cpp
output.c
project_structure.txt
README.md
syntax.md
build/                                ← 编译构建相关文件
  Pascal-Compiler.sln
  Pascal-Compiler.vcxproj
  CMakeCache.txt
  ...
include/
  codegen/
    CodeGenerator.h

```

```
    CodeGenUtils.h
    SymbolTable.h
lex/
    Lexer.h
    Token.h
semantic/
    Reduce.h
syntax/
    Ast.h
    Serialize.h
    SyntaxCheck.h
    SyntaxEntry.h
    SyntaxLl.h
    SyntaxLr.h
src/
    codegen/
        CodeGenerator.cpp
        CodeGenUtils.cpp
        SymbolTable.cpp
        test.cpp
    lex/
        Lex.cpp
        Lexer.cpp
        Token.cpp
    semantic/
        Reduce.cpp
    syntax/
        Ast.cpp
        Serialize.cpp
        SyntaxCheck.cpp
        SyntaxEntry.cpp
```

```
SyntaxLl.cpp
SyntaxLr.cpp
open_set/                                     ← 测试用例
    00_main.pas
    ...
    69_matrix_tran.pas
```

5.3 main 分支源程序清单

6 程序测试

6.1 正确的测试用例

- **测试集 1：调用标准过程**

测试功能：验证标准过程（如 `readln` 和 `writeln`）参数的构造与调用，涵盖整数、实数与数组元素的读写。

- **测试集 2：函数和过程的参数传递**

测试功能：验证函数和过程的参数传递机制，检查返回值的处理是否正确。

- **测试集 3：赋值语句**

测试功能：验证常量与变量的赋值运算是否正确，特别是数组元素和算术表达式赋值。

- **测试集 4：比较操作符**

测试功能：验证比较运算符（`=`、`<`、`<=`、`>=`、`>`）对整数和实数的适用性和准确性。

- **测试集 5：计算表达式**

测试功能：测试加、减、乘、除等算术运算的正确性。

- **测试集 6：For 循环**

测试功能：测试 `for` 循环结构，包括 `to` 和 `downto` 两种方向。

- **测试集 7：While 循环**

测试功能：测试 `while` 循环逻辑，确保循环条件成立时持续执行。

- **测试集 8: Repeat 循环**

测试功能: 测试 `repeat...until` 结构, 确保至少执行一次循环体。

- **测试集 9: 布尔控制的循环**

测试功能: 测试布尔值控制下的 `repeat` 与 `while` 循环。

- **测试集 10: 无限循环**

测试功能: 测试 `while true` 实现的无限循环行为。

- **测试集 11: 条件语句与递归函数**

测试功能: 测试 `if-then-else` 与递归函数的调用逻辑与返回值。

- **测试集 12: 条件语句 (if-then)**

测试功能: 测试条件判断与分支执行的正确性。

- **测试集 13: 变量与类型定义**

测试功能: 验证变量、常量、数组、`record` 类型的定义与声明的正确性。

6.2 错误检测与处理

- **测试集 1: 常量赋值错误**

错误类型: 试图修改常量值;

问题位置: `a := 3;`

错误信息: 常量不可修改。

- **测试集 2: 错误的常量定义符号**

错误类型: `const x == 10;` 使用了错误的 `==`;

正确应使用 `=`;

错误信息: 非法符号。

- **测试集 3: 变量声明缺少类型**

错误类型: `var a: ;` 中省略了类型;

错误信息: 变量类型缺失。

- **测试集 4: 赋值语句缺少表达式**

错误类型: `a := ;` 缺少右值;

错误信息: 不完整表达式。

- **测试集 5：表达式不完整**

错误类型：`a := 3 + ;` 缺少操作数；

错误信息：非法运算表达式。

- **测试集 6：拼写错误的类型**

错误类型：`integear` 应为 `integer`；

错误信息：未知类型。

- **测试集 7：函数缺少返回类型**

错误类型：`function AddNumbers(a, b: integer);` 缺返回类型；

错误信息：函数定义不完整。

- **测试集 8：过程体缺失**

错误类型：`procedure WriteNumber(a: integer);` 没有过程体；

错误信息：过程体缺失。

- **测试集 9：多个语法错误综合**

错误类型：拼写、赋值符号、缺少类型、函数返回类型缺失、非法赋值目标等。

错误位置：程序中多处，如 `poragam`、`const x == 2;`、`var c:;`、`set_a == val;` 等。

错误信息：多个语法错误，逐项报告。

7 课程设计总结

7.1 体会与收获

周正明

庞博

许恒栋

周宇洋

范兼玮

万云杰

在本次课程设计中，我主要负责项目的文档撰写工作。通过这个过程，我深入体会到技术文档在软件开发中的核心作用——它不仅帮助团队成员理解项目结构和工作流程，还能显著提升项目的可维护性和扩展性。

在文档撰写过程中，我重点掌握并应用了 L^AT_EX 排版工具，学习了文档结构组织、图表和代码环境插入等技术，为文档的专业性和规范性打下基础。这一技能也将为我未来在学术论文写作和技术报告编制中带来长远收益。

虽然我未直接参与代码的编写，但在整理和记录词法分析、语法分析、语义分析及代码生成等模块设计和测试的过程中，我对编译器构建的关键步骤有了更加深入的理解，也加强了与开发成员之间的技术沟通。

总的来说，这次课程设计不仅让我系统掌握了技术文档的编写技能，也让我深刻意识到沟通协作、文档规范和工具链在团队项目中的重要性。未来我将继续提升自己在技术写作和项目协作方面的能力，为参与更多复杂项目打下坚实基础。

7.2 设计过程中遇到或存在的主要问题及解决方案

7.3 改进建议